DOCUMENTATION ENGINEERING & TECHNICAL COMMUNICATION

Selected Documentation Work Samples

Dawna Joy Daludado
Maxio Help Center

> **Featured: October release-note redesign**
>
> I designed and completely overhauled the release-note page to make it more end-user friendly. Expandable “Learn more” sections explain why each change matters and where users can find it.

Included samples

1. October 2026 Product Release
2. Understand Wallets
3. Display Payment Terms on Invoices
4. RevOps.io
5. MCP Best Practices Guide
6. Troubleshoot Two-Factor Authentication

Captured October 28, 2026 (Pacific time). Public documentation may change after capture.
Portfolio: djdaludado-portfolio.vercel.app
Source content and product branding: Maxio.

October 2026 Product Release

Source: https://docs.maxio.com/getting-started/product-releases/october-2026-product-release • Selected entries; Learn more sections expanded.

Selling

Map more Salesforce fields and edit a transaction's Salesforce ID

Salesforce field mappings offer new options: the parent Account's name and the contact fields Salutation, Phone, Alt Phone, Fax, Contact, Alt Contact and Account Number as Account Metadata sources, a Customer's QuickBooks ID in Send Financial Records, and the renewal's Duration and Start Date for Renewal Opportunity Line Items. A Transaction's Salesforce ID can also be edited on its Update Transaction screen. (Released: 1 to 13 October 2026)

> **Learn more**
>
> **Why it matters:**
> You can map more of your Salesforce data to Maxio, and Maxio data back to Salesforce, without creating a Sales Order or filling fields in by hand, and any Editor can correct a Transaction's Salesforce ID after an Account merge.
>
> **Where to find it:**
> In the Salesforce integration settings, on the **Account Metadata Field Mappings**, **Send Financial Records** and **Send Renewal Opportunities** tabs, and on a Transaction's Update Transaction screen. For setup, see Configure Get Account Metadata for Salesforce, Configure Send Financial Records for Salesforce and Configure Send Renewal Opportunities for Salesforce.

Billing

Filter invoices by date and due date at the same time

Date, Due Date, and Paid Date are now separate filters on the Invoices list, so you can set any combination of them together instead of choosing only one. The redesigned filter bar also adds quick Document ID and Customer search, and a collapsible row of pill filters for the rest. (Released: 13 October 2026)

Learn more

Why it matters:

Previously you could filter Invoices by only one of Date, Due Date, or Paid Date at a time. Now you can combine them, for example Invoices issued last month with a due date this week, without exporting results and cross-referencing them yourself.

Where to find it:

Go to **Billing > Invoices** and click **Filter** to show the row of filters, where you can set a Date, Due Date, and/or Paid Date range. Document ID and Customer search stay available directly on the filter bar.

Payments

Refund under review notice for refunds held by Maxio Payments risk review

When Maxio Payments holds a refund for risk review instead of accepting or declining it immediately, Advanced Billing now shows a persistent **Refund Under Review** notice and tracks the hold, its release to the gateway, and its outcome in the invoice history. (Released: 14 October 2026)

Learn more

Why it matters:

Previously, a refund held for risk review looked like a plain failure, with no record of it in the invoice history and no way to tell a held refund from a rejected one. You now get a clear notice that stays visible until you dismiss it, and the invoice history shows the hold, its release, and whether it was ultimately approved or rejected.

Where to find it:

Refunding an invoice from its **More Options > Refund Invoice...** menu, or refunding a prepayment from the subscription's Prepayments tab, when Maxio Payments holds that refund for risk review.

Accounting

Import product families, products, payment profiles, payment terms and custom field definitions in bulk

Imports under **Admin** > **Import** can now create and update Payment Terms in bulk, create custom field definitions, Advanced Billing product families and products, and Advanced Billing payment profiles, leave a required custom field unmapped in an Update import, use dd/mm/yyyy or dd/mm/

yy as the Date Format, and preview an import's field mappings without uploading a file. Importing an Advanced Billing subscription that sets only Starts At to backdate it now fails validation on sites that are not invoice-centric billing, instead of importing without the backdate. (Released: 1 to 21 October 2026)

> **Learn more**
>
> **Why it matters:**
> You can load and correct records in bulk instead of entering them one at a time, bring stored payment methods across when you migrate to Advanced Billing, and see what an import needs before you build the file.
>
> **Where to find it:**
> Go to **Admin > Import** and choose Create, Update or Create & Update. Product family, product, payment profile and subscription imports for Advanced Billing appear only once Advanced Billing is connected and Maxio staff have turned the Advanced Billing importers on for your account. For the steps, see Run an Import, Format a CSV File for Import, Import Advanced Billing Payment Profiles and Import a Product Catalog from a CSV. Look for it here if it's available to you.

Insights

Add Salesforce Opportunity IDs to the Basic Report and get a warning before a report over 500,000 cells runs

The Basic Report's Additional Details offers a Salesforce Opportunity ID column for accounts with a Salesforce integration. A Basic Report or Projection Report that would contain more than 500,000 cells now stops before it generates results and shows a This report is too large to run warning with the rows, columns, cells and the limit. (Released: 6 to 21 October 2026)

> **Learn more**
>
> **Why it matters:**
> You can tie billing activity to its Salesforce Opportunity in the report, and you find out right away when a report is too large and what to change.
>
> **Where to find it:**
> On a Basic Report, open **Additional Details** and select **Transaction: Salesforce Opportunity ID**. The warning appears on the report's page with an **Edit report** button and a **Back to report history** button, and report emails include an **Edit the report** link.

Developer Experience

Filter customers, projects, sales orders, invoices and payments through the API and read customer numbers on includes

The Maxio Core API list endpoints accept new filters. Customers and Projects can be filtered by name, Sales Orders by customer identifiers, and Invoices, Payments and Customers by Advanced Billing IDs, invoice dates and Maxio IDs. The invoice, payment and contract lists honor a page size, invoice responses include three read-only fields, and the customer nested in invoice and transaction responses includes the customer's number. (Released: 12 to 15 October 2026)

> **Learn more**
>
> **Why it matters:**
> If your integration lists records, you can ask Maxio Core for exactly the records you need instead of paging through full lists and matching them yourself, and you can read a customer's number without a second call.
>
> **Where to find it:**
> In Maxio Core's in-app API documentation, under **Admin > Settings > API Documentation**, on the customers, projects, sales orders, invoices, payments and contracts list endpoints. A `page_size` above 100 is capped at 100, and a malformed Maxio ID returns an error. For an overview of the API, see Get Started with the Maxio Core API.

Understand Wallets

Source: https://docs.maxio.com/billing/catalog/wallets/understand-wallets

> BETA
>
> This feature is currently in beta and may change over time. During the beta period, functionality, availability, and performance may vary.
>
> To join the Beta program, request Beta access.

Entitlements define what a subscriber is eligible to use, and metering measures what they actually used. A wallet tracks what remains: a consumable balance on a single subscription, funded by grants, drawn down by usage, with every movement written to a ledger you can read back entry by entry.

Why use wallets

Use a wallet when an allowance is consumed rather than simply granted, and when you need to answer how much is left? as reliably as what were they entitled to? Wallets let you:

- Carry a running balance on a subscription instead of recalculating one from usage records.
- Fund that balance automatically on a recurring cadence, or add to it manually when a deal requires it.
- Expire unused value on a schedule you set, so allowances don't accumulate forever.
- Explain any balance by reading its ledger, where every grant, debit, expiration, and correction is a separate immutable row.

How wallets work

A wallet is provisioned from a usage-limit entitlement on a feature. You author the wallet behavior once on the feature, and every subscription that resolves that feature gets a wallet built to match.

What a feature is

A **feature** is one capability or allowance you define once in **Catalog > Features** and reuse across any Plan or Product. Each one carries a name, a stable **Handle** your code uses to reference it, and a **Feature Type** that decides how its value is read:

- **Access**: a capability that is either on or off, such as SSO or priority support.
- **Limit**: a numeric allowance, such as API calls or storage. Only a Limit feature can be wallet-backed.
- **Service Right**: a service level or tier that does not reset, such as an SLA.

Attaching a feature to a Plan or Product sets the value a subscriber receives, and that value resolves onto each subscription that holds it. See Understand Entitlements for how features, their kinds, and their values work in full.

Where a wallet comes from

Three pieces combine to produce a wallet on a subscription:

- **A usage-limit feature**: Wallet behavior can only be attached to a feature whose kind is usage limit. Access rights and service rights can't be wallet-backed.
- **A wallet product**: The feature's wallet behavior points at a billing product that is metered and has a unit configured. That product's unit denominates the wallet, so a balance is always expressed in tokens, credits, calls, or whatever unit you've defined.
- **A wallet on the subscription**: When the entitlement resolves, Advanced Billing creates the wallet against that subscription and funds it straight away, with no API call and no manual step. A wallet is always owned by exactly one subscription.

The configuration a wallet is built from is fixed once the wallet exists, so a subscription's wallet keeps behaving the way it did when it was created even if you later change the feature.

A wallet can also be created manually on a subscription, for cases the feature's own configuration doesn't cover. See Add a Wallet to a Subscription.

What makes up the balance

A wallet's balance isn't a single stored number you edit. It's the sum of what remains across the wallet's grants. Each grant records the amount it was originally worth, how much of it is still unspent, and, optionally, when it expires.

Because the balance is made of grants, usage has to draw down from somewhere specific. Advanced Billing spends grants in a fixed order: grants that carry an expiration date go first, soonest expiration first, and grants that never expire are spent last. That means value at risk of being written off is always consumed before value that remains available next month.

How the balance changes

Every change to a wallet falls into one of three shapes.

Grants add value

A grant is value arriving in the wallet. Grants come from five places:

- **Recurring**: Written automatically on the refill cadence set on the feature.
- **One-time**: Added deliberately, for example to compensate a customer after a service incident.
- **Initial balance**: The opening balance the wallet was created with.
- **Allowance increase**: Written automatically when the funding feature's allowance is raised mid-period.
- **Purchase**: Bought through the **Purchase** action on the subscription's **Wallets** tab, and billed on an invoice.

A grant's original amount is fixed at creation and can't be edited afterward. Correcting a grant means posting new ledger activity, not rewriting the grant.

Usage draws it down

Usage produces debits against the wallet, spent across grants in the drain order described above.

For a wallet paired with a meter, the balance you see also accounts for usage the meter has recorded but the ledger hasn't yet settled. Advanced Billing reads the meter's usage, converts it into the wallet's unit, and shows you the balance net of that usage. If the usage store can't be read at that moment, the recorded balance is shown instead and marked as out of date, rather than reporting a number that might be wrong.

That metered usage is also written to the ledger in its own right: when a grant is fully spent, when a grant expires, and periodically in between. The ledger therefore catches up with the balance rather than trailing it indefinitely, and **Balance Breakdown** on the wallet's page already counts that unposted usage against each grant. While the balance is current, the grants' left amounts add up to the balance shown above them, unless the balance is below zero.

Where a wallet discards overflow, usage a meter records while the balance is already at zero is never written to that wallet's ledger. It stays recorded in the meter, and it isn't added later even if the wallet is funded again.

Expiration writes off what remains

If the feature sets a grant expiration policy, each grant is stamped with an expiration date when it's written. Once that moment passes, whatever is left in the grant is written off and recorded in the ledger as an expiration.

> Refills and expiration write-offs are processed on a schedule rather than the instant they come due, so a balance can briefly lag the moment a refill or expiration was scheduled for.

What happens when a wallet runs out

Once the balance is gone, further consumption isn't silently absorbed. What happens to it depends on what the wallet was set to do when usage exceeds the balance, a choice fixed when the wallet is created:

- **Discard**: Consumption beyond the available balance is refused. The balance stops at zero and doesn't go negative.
- **Allow negative**: Consumption beyond the available balance is still recorded, and the balance goes below zero.
- **Bill as overage**: The balance stops at zero, and consumption beyond it is recorded as overage and billed on the subscription's invoice.

> **Important:** Allowing negative balances means a subscriber can consume past their allowance without being invoiced for it. A negative balance is repaid in units by the next refill, grant or purchase. If you want consumption past the allowance charged, use Bill as overage instead.

How overage is billed

On a wallet set to bill overage, usage that the wallet's grants can't cover is posted to the ledger as an **Overage** row, and the balance stays at zero. That overage is billed in arrears, as one charge for the period's overage at the wallet product price point the wallet stores. When it is billed follows that price point's own billing interval:

- **A billing interval**: a monthly interval bills each month's overage at the month's end, even on a subscription with an annual plan.
- **No billing interval**: the overage is billed on the subscription's renewal invoice.

The quantity is in the wallet's own unit and can be fractional, and a period with no overage adds no overage line to the invoice.

The overage line takes the wallet product's name as its title. When the line bills a single wallet that was created from a feature, the title also says how much the subscription's allowance included, for example **Additional calls (beyond 100 included)**. The same title appears on the invoice and on previews of the upcoming invoice.

The included amount is the allowance of every refill schedule still funding the wallet, added together. Two cases change what the title quotes:

- If an allowance is lowered during the period, the title quotes what was already granted for that period.
- On the final invoice of a subscription canceled at the end of its period, the title still quotes the allowance that funded the wallet during that period.

The overage line keeps the plain wallet product name when:

- The wallet was added manually on the subscription rather than created from a feature.
- The line bills more than one wallet, including overage from an archived wallet that hasn't been invoiced yet.
- The included amount can't be read from the wallet's refill schedules, or adds up to zero.

These rules apply to overage billing:

- **The price point has to be able to bill it.** A price point with hybrid pricing can't bill overage. On a termed subscription in a currency other than your site's primary currency, the price point needs a definitive price in that currency.
- **One price point per wallet product on a subscription.** If a subscription already bills overage for a wallet product at one price point, a second overage wallet on the same wallet product has to use that price point too.
- **Usage is never billed twice.** An overage wallet and an event-based component can't bill the same meter on one subscription, whichever is added first, including a component scheduled to activate at a renewal or through a pending change. Wallets that discard or allow a negative balance aren't affected.
- **Several overage wallets can share a meter.** When several overage wallets on one subscription read the same meter, usage that all of them are over on is billed once, on the wallet whose price is lowest per meter unit. The other wallets post a zero-amount row whose memo reads **Overage of** the amount **deferred to wallet** followed by that wallet's name.
- **A cancellation at the end of the period bills the last overage.** The final renewal invoice bills the overage through the end of the period, including usage from the wallet's last usage window. Usage after that isn't captured as overage.
- **An immediate cancellation depends on your site.** If your site charges unbilled usage after a subscription is canceled, overage up to the end of the canceled subscription's period is billed with that unbilled usage, last usage window included. If it doesn't, overage capture stops when the subscription is canceled, and overage captured before then that isn't on an invoice yet is written off. Either way, usage after the cutoff isn't captured as overage.
- **Reactivation writes off what's unbilled.** Reactivating a subscription doesn't bill overage from before the reactivation. Any overage not yet billed is written off.
- **The wallet product stays in use.** While an active wallet bills overage through it, the wallet product can't be archived (**Wallet product is in use by an active overage wallet**), and neither can the price point the wallet bills at (**Price point is in use by an active overage wallet**).

If overage can't be billed when usage is recorded, the part beyond the balance isn't recorded. Usage you record yourself reports that the amount couldn't be billed as overage and wasn't recorded. For a metered wallet, the ledger gets a zero-amount row whose memo starts **Overage of** the amount **not billed:** followed by the reason.

How the ledger records every change

Every movement in a wallet is a row in its ledger, and each row carries the balance as it stood immediately after that movement. Rows are one of six kinds:

Kind	What it records
Grant	Value added by a recurring refill, an allowance increase, a one-time grant, or a purchase.
Initial balance	The opening balance the wallet was created with.
Usage	Value drawn against the wallet.
Expired	Unspent value written off when a grant expired.
Adjustment	A correction posted against an earlier entry.
Overage	Usage beyond the balance on a wallet that bills overage, to be billed on the subscription's invoice.

Ledger rows are immutable once written. Nothing is ever edited or deleted, so correcting a mistake means posting an adjustment that references the original entry. The history of what actually happened stays intact, and the correction is visible alongside it.

Example: a subscriber is granted 20M tokens at renewal and uses 5.5M of them. The ledger shows a grant row of +20M and a debit row of -5.5M, with a balance of 14.5M after the second row. If that 5.5M turns out to have been over-counted, an adjustment row is posted against the debit rather than the debit being changed, and the ledger shows all three entries.

Requirements, dependencies, and limitations

Review these before using wallets in production, because they affect which sites can use wallets at all and what you need to set up first:

- Wallets must be enabled for your site, and the site must be on invoice-centric billing. Contact your Customer Success Manager or Maxio Support to request activation.
- Drawing a wallet down from metered usage requires **metered wallet consumption**, a further setting on top of wallets being enabled. Ask your Customer Success Manager to enable it if you intend to pair wallets with meters.

- The billing product that denominates a wallet must be metered and must have a unit configured. Create one in **Catalog > Wallet Products** and give it a price point before you author wallet behavior on a feature; see Create and Manage Wallet Products.

What wallets can't do and how they behave

These limits and behaviors apply once wallets are running, and they affect where wallets can be used, how quickly balances settle, and what you can change afterward:

- Wallet behavior can only be attached to a feature whose kind is usage limit. That feature also can't be attached to a Plan or Product that's metered, prepaid-usage, or events-based, because those already meter their own consumption and a usage-limit feature there would double it. Attaching one is refused with a 422, whether or not the feature is wallet-backed.
- Wallets aren't available on a prepaid subscription, since a prepaid subscription already meters against its own balance. A wallet-backed feature isn't provisioned one, the **Add Wallet** wizard isn't offered, and `POST /wallets.json` refuses with "Wallets are not available on prepaid subscriptions".
- A subscription that hasn't gone live yet (Awaiting Signup, Awaiting Signup Date, Pending, or Failed to Create) isn't provisioned a wallet at all, not even at a zero balance. As soon as it goes live, the wallet is created and funded for the period already in progress, with no backlog from the waiting period to catch up on.
- A wallet belongs to exactly one subscription. Wallets can't be shared or pooled across subscriptions, though a subscription can hold more than one wallet if it carries more than one funding feature.
- Refills and expiration write-offs are processed on a recurring schedule, so ledger settlement isn't instantaneous.
- A grant's original amount can't be changed after the grant is written, and ledger rows can't be edited or deleted.
- A wallet created manually supports only the **Units** balance type. **Monetary (USD)** balances, and drawing a wallet down by subscription-product charges, appear in the Add Wallet wizard but aren't available yet.
- Choosing **Manual** under **Drawn down by** saves nothing. It describes how a wallet with no meter already behaves rather than configuring anything, and funding is unchanged either way.
- Pairing a wallet with a meter is permanent. The meter and the conversion rate are both fixed when the wallet is created and can't be changed, removed, or re-picked afterward. Only meters that sum or count usage can be paired; every other meter is listed disabled, with the reason shown.
- A wallet's opening balance and any manual grant can each carry their own expiration date. Grants written by a feature's refill cadence take their expiration from that feature's **Grants Expiration** policy instead.
- On a wallet's details page, the ledger rows, the metered usage window, the line that says how current a metered balance is, and an adjustment's entry date resolve in your site's time zone and carry its abbreviation, so they read the same for everyone regardless of their own browser's time zone. A grant's expiration date is a calendar date judged in your site's time zone, with no time or abbreviation. The **Last activity** date on a wallet card in the **Wallets** tab's list uses your browser's time zone.

- A wallet that allows a negative balance never invoices the shortfall. It's repaid in units by the next refill, grant or purchase. To charge for consumption past zero, create the wallet to bill overage instead (see How overage is billed), or sell more balance with the **Purchase** action on the subscription's **Wallets** tab.

See Configure a Wallet to author wallet behavior on a usage-limit feature so subscriptions receive wallets automatically.

See Manage Subscription Wallets to add grants, buy more units, record usage, and archive a wallet.

See Troubleshoot Wallets to work out why a balance, a refill, or a correction isn't behaving as expected.

See Manage Wallets with the API to do the same work from your own application.

Display Payment Terms on Invoices

Source: https://docs.maxio.com/billing/invoicing/branding-themes/display-payment-terms-on-invoices

Show the payment term name that is set on an Invoice in Maxio Core, such as "Net 30" or "15th of next month", on Invoices generated from a Branding Theme. This is different from Net Terms, which only expresses a day count and can't capture a day-of-month term; Payment Terms carries the name from Maxio Core as it is.

Before you begin

A few things to confirm before you add Payment Terms to a theme:

- Your Site needs invoice-centric billing enabled and at least one Branding Theme. See Understand Branding Themes.
- Payment Terms only fills in on Invoices that come from Maxio Core with a payment term set on the Invoice itself. It does not use the Customer's or contract's default term, so an Invoice with a due date set manually has no payment term.
- Advanced Billing doesn't calculate or assign a payment term on its own, so an Invoice Advanced Billing creates natively always shows this field blank, by design, with no fallback value.
- Net Terms and the Due Date are unaffected by Payment Terms and continue to work the same way. See Understand Net Terms.

Show payment terms in the document details block

The Document Details block has a Payment Terms row. It is off by default, so existing themes don't change until you turn it on.

To add the Payment Terms row to an Invoice layout

1. Go to **Config > Branding**, click **Actions** on the theme, then click **Template Editor**.
2. Click the Document Details block on the Invoice layout.
3. In the **Field Layout** section, find **Document Details Order** and turn on **Payment Terms**.
4. Click **Update Template** to save.

The row shows the label, "Terms" by default, and the term name Maxio Core sent, for example "Net 30". The row is left out on an Invoice with no payment term, so it never shows an empty label.

Add the merge tags to your own text

To show the term somewhere else on the Invoice, add the **Payment Terms Label** and **Payment Terms** merge tags to a text block. Both are in the **Document Details** group of the merge tag list. In the editor preview they show the sample values "Terms" and "Net 30".

To hide that text when there is no term, wrap it in **Has Payment Terms**, which shows the text only when the Invoice has a payment term. This also lets a single theme serve both Core-synced and Advanced-Billing-native Invoices: the Payment Terms text only appears on the Invoices that actually carry a term.

Customize the label text

The default label is "Terms". To use different wording, set a Custom Translation for **Payment Terms Label**. See Customize Invoice and Credit Note Text.

RevOps.io

Source: https://docs.maxio.com/selling/revopsio

Help Center › Selling › RevOps.io

Selling

RevOps.io (Maxio CPQ) is where you build quotes, route them for approval, and collect e-signatures. When a deal is signed, the Maxio integration creates or updates the matching Sales Order in Maxio. With Sales Order Auto-Process on, it is processed into subscriptions and transactions; otherwise it waits in the Unprocessed Sales Orders queue.

··View as Markdown·

Most links on this page open docs.revops.io

RevOps.io has its own documentation site. Links to RevOps.io articles on this page open docs.revops.io. Go to RevOps Documentation (opens docs.revops.io)

How the pieces fit together

Every RevOps.io deal moves through three stages: you build it, it gets approved, then it's signed and synced to your other systems.

Build the deal
Templates (opens docs.revops.io)Every deal starts from a template with your usual terms and line items.SKUs (opens docs.revops.io)Your products and pricing models, from per-user to tiered, volume, and usage.Terms (opens docs.revops.io)The legal conditions of the agreement, including built-in and custom terms.
Get it approved
Approval workflows (opens docs.revops.io)Approve qualifying deals automatically, or route them to the right approvers.Deal status (opens docs.revops.io)Deals move from Draft through approval to signed. Material changes can send them back.Functions (opens docs.revops.io)Custom TypeScript for validations and approval triggers.
Signed and synced
eSignature (opens docs.revops.io)Collect signatures with RevOps.io's own e-signature or with Docusign.CRM sync (opens docs.revops.io)Connect Salesforce or HubSpot to import deal data and sync results back.Maxio integrationSigned deals create Sales Orders in Maxio. With Auto-Process on, they become subscriptions (Advanced Billing only) and transactions.

Start with what you're trying to do

Pick the goal closest to yours. Each path lists articles in the order to read them.

- **Get started with RevOps.io**

  Onboard your team and create your first deal.

  1. Quick Start Guide (opens docs.revops.io)
  2. How do I create a Deal? (opens docs.revops.io)
  3. Setting up Templates (opens docs.revops.io)

- **Build your product and price list**

  Create SKUs one at a time or import them from a spreadsheet.

  1. SKUs & Line Items (opens docs.revops.io)
  2. Bulk Import SKUs (opens docs.revops.io)
  3. How to Create a Per-user SKU (opens docs.revops.io)

- **Automate deal approvals**

  Approve deals automatically or route them to the right people.

  1. Creating Workflows (opens docs.revops.io)
  2. Auto-Approval Workflows (opens docs.revops.io)
  3. When do Workflows run? (opens docs.revops.io)

- **Send a deal for signature**

  Share it with your customer and add countersigners.

  1. Sharing deals with customers (opens docs.revops.io)
  2. Managing Signature Type and Order (opens docs.revops.io)
  3. Countersigning RevOps Deals (opens docs.revops.io)

- **Connect your CRM**

  Link deals to Salesforce Opportunities or HubSpot deals.

  1. CRM Integrations (opens docs.revops.io)
  2. Salesforce (opens docs.revops.io)
  3. HubSpot (opens docs.revops.io)

- **Send signed deals to Maxio**

  Create Sales Orders when a deal is signed, and have them processed automatically with Auto-Process.

  1. Setup RevOps.io Integration
  2. Processing Sales Orders

- **Quote renewals and expansions**

  Set up templates for renewals and mid-term additions.

  1. Setting up Renewal Templates (opens docs.revops.io)
  2. Setting up a Cotermed Expansion Template (opens docs.revops.io)
  3. Prorating Deals (opens docs.revops.io)

Answers to common questions

The RevOps.io questions customers ask most, and where to find the answer.

- What can a Sales Rep see and do in RevOps.io?Managing User Roles & Permissions → (opens docs.revops.io)
- How do I export my SKUs?Export SKUs → (opens docs.revops.io)
- How do new SKUs get into Salesforce?Managing SKUs with Salesforce → (opens docs.revops.io)
- Why did my deal go back to Draft?Managing Deal Status → (opens docs.revops.io)
- When will my customer be notified?When will my customer be notified? → (opens docs.revops.io)
- How do I redline a deal?Export RevOps Deals as DocX files → (opens docs.revops.io)

After your deals are signed

Sales Orders →Process the orders your signed deals create.Salesforce for Maxio Core →Connect Salesforce to Maxio Core.HubSpot →Connect HubSpot to Advanced Billing.

Didn't find what you need? Contact Support

MCP Best Practices Guide

Source: https://docs.maxio.com/configuration/maxio-mcp-server/mcp-best-practices-guide

Finance, revenue, and billing teams get the most from the Maxio Model Context Protocol (MCP) by treating it as a governed reasoning layer rather than an automation tool. Deciding where it belongs means knowing which tasks suit reasoning over live data and which still need the repeatability of an API.

The four layers of AI-enabled finance and billing workflows

AI touches finance work at four levels of access, each with a different trade-off between reasoning and reliability. Knowing which layer a task belongs to is what stops you reaching for the wrong tool.

Layer 1: conversational AI, with no live system access

Useful for thinking, not for answering questions about your own numbers.

- Drafting memos and summaries
- Explaining finance or billing concepts
- Brainstorming KPI frameworks

Limitation: No access to real-time company data.

Layer 2: tool-enabled AI through MCP

This is where Maxio MCP sits, and where live data enters the conversation.

- Financial and billing investigations
- Executive-ready summaries
- Diagnostic analysis
- Assisted, human-reviewed execution

Combines machine-speed data retrieval with contextual reasoning, narrative explanation, and prioritization.

Layer 3: reusable AI workflows from templates

The same MCP access, but with the prompt written once and re-run.

- Monthly collections reviews

- Quarterly billing and ARR summaries
- Renewal risk assessments
- Invoice variance analysis

Benefit: Consistency and faster execution.

Layer 4: programmatic integrations through APIs

No model in the loop, so the output is identical every run.

- Billing automation
- Revenue recognition pipelines
- Invoice processing
- System synchronization

Benefit: Reliability, auditability, and scale.

Choosing between MCP and APIs

The two are not competing. The question is whether a task needs judgment or needs repeatability.

Use MCP when you need reasoning

Reach for MCP when the shape of the answer is not known in advance.

- Questions are not fully defined upfront
- You need judgment, ranking, or narrative explanation
- Analysis evolves interactively
- Outputs are reviewed by a human

Example use cases:

- Cash flow and collections prioritization
- Subscription momentum and ARR bridge analysis
- Renewal risk triage
- Invoice and payment anomaly investigation

Use APIs when you need reliability

Reach for an API when the same input must always produce the same output.

- Processes must run identically every time
- Outputs must be structured and auditable
- No human is in the loop
- Workflows are high-volume and recurring

Rule of thumb: MCP + AI = flexible reasoning over live data AI + APIs = structured, repeatable automation

A CFO's mental model for MCP

MCP should be viewed as a financial and billing analyst with governed system access.

- Pulls financial and billing data
- Connects context across systems
- Explains what changed and why
- Recommends prioritized next steps

It does not replace

The analogy has limits, and these are where it breaks down.

- Core billing and accounting systems
- Deterministic workflows
- Audit-controlled processes
- Human accountability

Troubleshoot Two-Factor Authentication

Source: https://docs.maxio.com/configuration/authentication-access/troubleshoot-two-factor-authentication

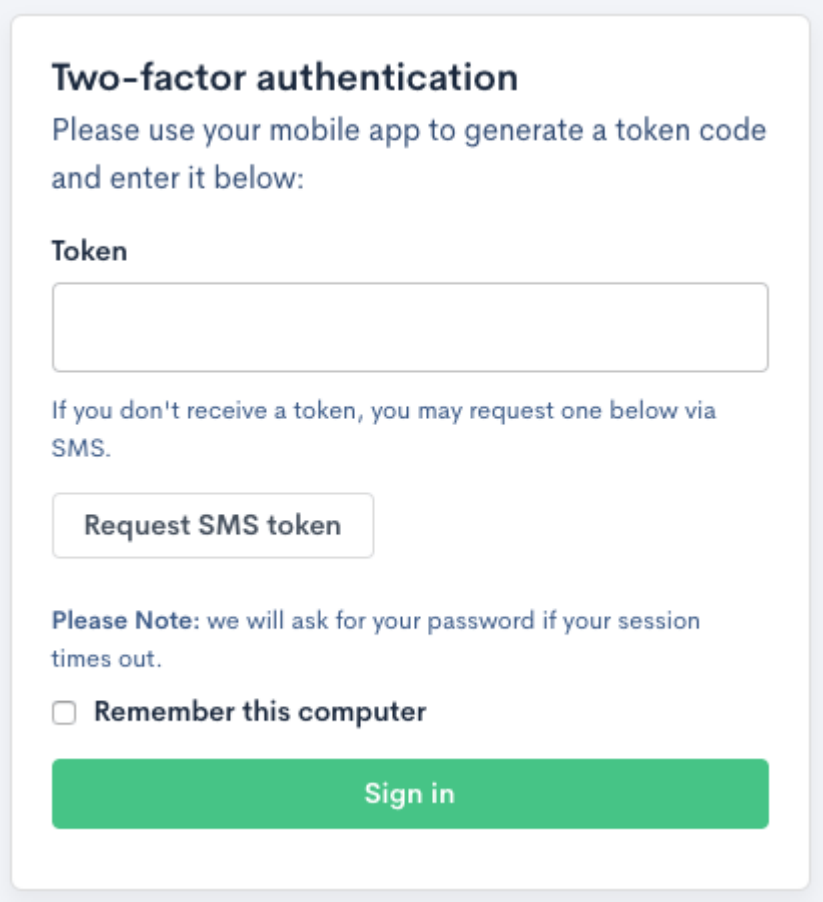
Two-Factor Authentication (2FA) can block your sign-in for reasons that have nothing to do with your password. The recovery options open to you depend on which Maxio product you are signing in to, because each one presents a different verification screen.

I did not receive my verification code

Start by confirming your phone has service and can receive text messages, then wait a moment and try again. Codes sometimes arrive late rather than never.

If the code still does not arrive, use whichever fallback your verification screen offers.

In Advanced Billing, select **Request SMS token** to send a new code. This button appears only if your account has SMS fallback turned on, so it is not on every Advanced Billing sign-in screen. An administrator turns SMS fallback on from the **Security** page in Advanced Billing, as described in the Configure Security Settings help article.



The Advanced Billing verification screen, with SMS fallback available

In Maxio Core, the verification screen lists your other registered devices under "Or, alternatively, use one of your backup devices:" and offers a **Use Backup Token** button under "As a last resort, you can use a backup token:".

I lost my phone or cannot use my authentication device

Use a backup method you set up earlier. In Maxio Core, that means another registered device or **Use Backup Token**, using one of the backup tokens Maxio issued when you enrolled. Each backup token works once.

If you have no backup method left, you cannot restore access yourself. An administrator on your account can remove 2FA from your user record, which clears every device registered to you and lets you enroll again from scratch. Ask them to do that, and if nobody on your account can, contact Contacting Support.

Maxio asks for a code every time, even on a computer I trust

In Advanced Billing, selecting **Remember this computer** on the verification screen stops Maxio asking for a code on that computer for **one week**. After a week the trust expires and you are challenged again, which is expected rather than a fault.

You are also challenged again sooner if you clear your browser cookies, switch browsers, or sign in from a different computer, because the trust is stored as a cookie on that one browser.

Why does Maxio ask me to set up 2FA?

An administrator on your account either enabled 2FA for you or required it for everyone. Until you finish enrolling, Maxio returns you to the setup screen instead of letting you continue.

I use Enterprise SSO

Your identity provider handles second-factor verification, not Maxio, so none of the steps above apply to you. Contact your internal IT team for help.